ChurnMaster: Learning Routing-Table Maintenance for a Kademlia-like DHT under Churn with Tabular Q-Learning

Deepansh Singh Ankur Kumar

AI2 Course Mini-Project · Code: `https://github.com/Ankurkumar359/ChurnMaster` · Release tag: `v1.2.0`

1 Introduction

In a Kademlia DHT [1], each node keeps up to k contacts per XOR-distance bucket. Under churn, contacts go offline, and the node must decide when to ping a contact and when to replace it with a cached candidate. Pinging costs messages, whereas dead entries cause lookup timeouts. Standard Kademlia pings the least-recently-seen contact whenever a newcomer is offered to a full bucket; this rule is robust but message-heavy.

Objective. We ask whether a tabular Q-learning policy [2, 3] that uses only locally observable signals can trade maintenance traffic against lookup success and latency better than (a) Kademlia-style maintenance, (b) a random policy with the same action frequencies, and (c) a tuned rule-based heuristic, and whether it transfers to an unseen churn level. Hypotheses H1–H5 were pre-registered and are reported whether or not they pass.

2 RL Formulation

2.1 Environment

The environment is a deterministic, pure-Python discrete-event simulator of a 128-node DHT with 8-bit identifiers, which gives the focal node F eight buckets. Each bucket holds $k = 4$ contacts and a four-entry replacement cache (TTL 300 s); lookups use parallelism $\alpha_{\mathrm{L}} = 2$ and are served by the 3 closest online replicas. Round-trip times are log-normal and packets are lost with probability 0.02. Only F adapts; all other nodes rebuild stale routing tables every 300 s. Lookups arrive at 1/s with uniform keys and run atomically with a 2000 ms deadline. An episode consists of 600 s of warm-up and 2400 s of measurement, with one decision epoch every 30 s. Each node alternates Weibull-distributed sessions (shape 0.6) with exponential downtimes. Policy-independent randomness (loss, jitter, table rebuilds) is keyed by event identity, so different policies observe identical outcomes for identical messages.

2.2 State Representation

Agent observations. At every epoch, F makes one decision per non-empty bucket. A decision concerns that bucket's least-recently-seen *incumbent* contact and its freshest cached *candidate*, if any.

Observable features. The agent observes seven features of this pair, each discretized into two or three bins (Table 1); a value exactly on a threshold falls into the higher bin. SRTT is the incumbent's smoothed round-trip time.

Table 1. The seven discretized state features.

Feature (bins)	Categories or thresholds	Meaning
Tier (3)	top, second, other	Bucket $b = 7$, $b = 6$, or $b \le 5$
Failures (3)	0, 1, ≥ 2	Consecutive failed pings to incumbent
Last reply (2)	< 60 s, ≥ 60 s	Time since incumbent last replied
Age (3)	< 120, 120–1200, ≥ 1200 s	Incumbent's time in the table
Candidate (3)	none, fresh (< 30 s), stale	Freshest cached; stale is 30–300 s
Faster (2)	yes, no	Cand. RTT $< 0.7\times$ incumbent SRTT
Churn (2)	< 0.10, ≥ 0.10	F's timeout share, last 120 s

State-space size. The state index is the mixed-radix combination of the bins,

$$|\mathcal{S}| = 3 \times 3 \times 2 \times 3 \times 3 \times 2 \times 2 = 648. \tag{1}$$

Only 540 of these indices are reachable, because "candidate faster" can be *yes* only when a candidate exists, so the 108 combinations of "no candidate" with "faster" never occur.

Partial observability. The full network state, including which nodes are online, their types, congestion and other nodes' tables, is hidden from F. The agent therefore acts on local bookkeeping only. Treating the discretized observation as a Markov state is an approximation to a partially observable problem [4], so the convergence guarantees of Q-learning do not apply and all claims below are empirical.

2.3 Actions

For each decision the agent chooses one of three actions. KEEP sends no message. PROBE pings the incumbent; if the ping fails, the incumbent is evicted and the candidate is tried. REPLACE pings the candidate and swaps it in if it answers; it is masked when no candidate exists. Empty slots are filled from the cache at no cost.

2.4 Reward and Score

For a measured lookup i with success indicator $S_i \in \{0, 1\}$, latency L_i (failed lookups count at their give-up time) and deadline $D = 2000$ ms, the per-lookup loss is

$$\ell_i = (1 - S_i) + w_L \min\left(\frac{L_i}{D},\, 1\right). \tag{2}$$

Let $\mathcal{L}_{b,k}$ be the lookups whose key falls in bucket b during epoch k, and let $P_{b,k}$ and $E_{b,k}$ be the pings and evictions caused by the decision for (b, k). The reward of that decision is

$$r_{b,k} = -\sum_{i \in \mathcal{L}_{b,k}} \ell_i \;-\; \big(w_M P_{b,k} + w_R E_{b,k}\big), \tag{3}$$

with weights $w_L = 0.3$, $w_M = 0.1$ and $w_R = 0.15$. Over an episode with N measured lookups, P pings and E evictions, the evaluation score is

$$J = 1 - \bar{\ell} - \frac{w_M P + w_R E}{N} = 1 + \frac{1}{N}\sum_{b,k} r_{b,k}, \tag{4}$$

where $\bar{\ell}$ is the mean of (2). The identity in (4) held on every diagnosed episode (maximum error 3.3×10^{-16}).

2.5 Tabular Q-Learning

The action-value function Q is a 648×3 table initialised to zero. Each one-epoch bucket transition (s, a, r, s'), with s' the same bucket's state at the next epoch, triggers the update

$$Q(s,a) \leftarrow Q(s,a) + \alpha_n\Big[\, r + \gamma \max_{a' \in \mathcal{A}(s')} Q(s', a') - Q(s,a)\Big], \tag{5}$$

where $\mathcal{A}(s')$ is the set of allowed actions in s', γ is the discount factor, and the step size $\alpha_n = \max\big(\alpha_{\min}, (1+n)^{-0.7}\big)$ depends on the number n of earlier updates of (s, a). Transitions that span an empty bucket or the episode end are dropped. Exploration is ϵ-greedy, with ϵ decaying linearly from 1.0 to 0.05 over the first 60% of episodes; greedy ties break in the order KEEP, PROBE, REPLACE. A grid search on validation seeds selected $\gamma = 0.8$ and $\alpha_{\min} = 0.02$.

3 Experimental Setup

Scenarios. Nodes are either stable or flaky. *stable* has 70% stable nodes, mean sessions of 4 h (stable) and 30 min (flaky), and 20 min downtimes; *high_churn* has 30%, 1 h/5 min sessions and 10 min downtimes; *held-out_medium* has 45%, 2 h/10 min sessions and 12 min downtimes. The last scenario is never used for training or tuning.

Training. We train 3 independent runs of 200 episodes each, alternating stable and high_churn episodes, and validate every 20 episodes on 5 separate seeds per training scenario.

Baselines. *kademlia_epoch* probes whenever a candidate exists, applied at the shared 30 s decision points. *matched_random* samples actions with RL's own frequencies, which tests whether *where* RL acts matters. *tuned_heuristic* is the best of 32 rule configurations on training seeds; RL's table can represent every configuration exactly. The evaluation also includes *never_replace*, *always_replace*, *uniform_random*, real-protocol *kademlia_std*, the default heuristic, a privileged per-scenario heuristic and a ground-truth *oracle* (a reference, not an upper bound).

Table 2. Primary evaluation: means over 20 evaluation seeds (RL is the mean of 3 training runs). Higher J and lookup success (succ., %) are better; lower p95 latency (ms, all lookups) and pings per hour are better. Bold: best of the four policies per J, p95 and pings column. ΔJ is RL minus comparator with a 95% bootstrap CI; all nine differences are significant after Holm correction. Source: `results/eval/`.

	stable				high_churn				heldout_medium			
Policy	J	succ. %	p95	pings/h	J	succ. %	p95	pings/h	J	succ. %	p95	pings/h
Kademlia-epoch	0.9642	100.00	371	383.9	0.9619	99.66	541	338.0	0.9640	99.96	466	368.7
Matched random	0.9589	99.54	644	8.0	0.9578	99.69	730	48.3	0.9603	99.43	644	26.1
Tuned heuristic	**0.9760**	100.00	**371**	9.3	**0.9735**	99.84	**468**	**38.5**	**0.9749**	99.96	**379**	**20.4**
Q-learning (RL)	0.9739	100.00	457	**7.1**	0.9705	99.79	554	43.8	0.9729	99.96	521	22.6
ΔJ, RL − Kademlia-epoch	+0.0097 [+0.0091, +0.0102]				+0.0085 [+0.0065, +0.0110]				+0.0089 [+0.0081, +0.0098]			
ΔJ, RL − Tuned heuristic	−0.0021 [−0.0030, −0.0013]				−0.0030 [−0.0044, −0.0017]				−0.0020 [−0.0027, −0.0013]			
ΔJ, RL − Matched random	+0.0149 [+0.0061, +0.0295]				+0.0127 [+0.0100, +0.0160]				+0.0126 [+0.0053, +0.0235]			
RL ping change vs. Kademlia-epoch	−98.1%				−87.0%				−93.9%			

Table 3. Pre-registered hypotheses and final verdicts (source: `hypotheses.csv`).

ID	Statement	Verdict
H1	RL beats matched random on J, every scenario (Holm)	pass
H2	high_churn: RL non-inferior (1 pp success) to Kademlia-epoch and tuned heuristic, and better on ≥ 1 of success, p95, pings	inconclusive
H3	Pings ≤ Kademlia-epoch; p95 ≤ 1.05× best baseline (all)	fail
H4	H2 holds on heldout_medium	inconclusive
H5	All 3 RL runs agree with pooled direction (high_churn)	pass

Evaluation protocol. The 20 evaluation seeds per scenario were locked before evaluation and checked against 635 other seed states for overlap (none). The evaluation ran once: 780 episodes (13 policy labels × 3 scenarios × 20 seeds), where the 13 labels are the 10 baselines above plus the 3 independently trained RL runs. RL denotes the per-seed mean of these 3 runs. Comparisons are paired by seed, with 95% percentile-bootstrap confidence intervals (10 000 resamples) and Wilcoxon signed-rank tests. The 9 primary tests (RL versus *kademlia_epoch*, *tuned_heuristic* and *matched_random*, in each scenario, on J) use Holm correction [5]. The environment hash is frozen before training, and an import guard and a counterfactual test prevent RL and every non-oracle policy from reading ground truth.

4 Primary Results

Table 2 and Fig. 1 summarise the locked evaluation. All 9 primary tests are significant after Holm correction ($p_{\text{Holm}} \le 4.8 \times 10^{-4}$). Differences are RL minus comparator, so a negative ΔJ means that RL scores lower.

- **RL improves on Kademlia-style maintenance.** RL achieves higher J in all three scenarios (ΔJ = +0.0085 to +0.0097), including the unseen heldout_medium, while sending 87–98% fewer *maintenance pings per hour* (338–384 versus 7–44). Lookup success barely changes (99.79–100.00% for RL): in high_churn, RL − Kademlia-epoch is +0.13 pp [−0.04, +0.35] and RL − tuned heuristic is −0.05 pp [−0.16, +0.05]. Fewer pings do not imply fewer total messages: lookup queries dominate, and F's messages per lookup fall by only about 3% (derived from `per_seed.csv`); other nodes' traffic is not measured.
- **The gain is not explained by acting less often.** RL beats *matched_random*, which uses the same action frequencies, by ΔJ = +0.013 to +0.015 (H1 pass); all three runs agree with the pooled direction (H5 pass).
- **The tuned heuristic performs better than RL.** RL has significantly lower J (ΔJ = −0.0020 to −0.0030) at a similar ping rate, and its p95 latency is 18–38% above that of the best eligible baseline (H3 fail; ratios 1.26, 1.18 and 1.38). H2 and H4, which require non-inferiority and a significant gain over both baselines, are inconclusive. RL's p95 latency is also higher than Kademlia-epoch's in stable and heldout_medium (+85 and +55 ms); in high_churn the difference is not significant.

5 Exploratory Ablations

These ablations are *not* part of the primary evaluation. They were run post hoc on separate TEST seeds (disjoint from the evaluation seeds) without multiple-comparison correction; they suggest hypotheses but confirm none.

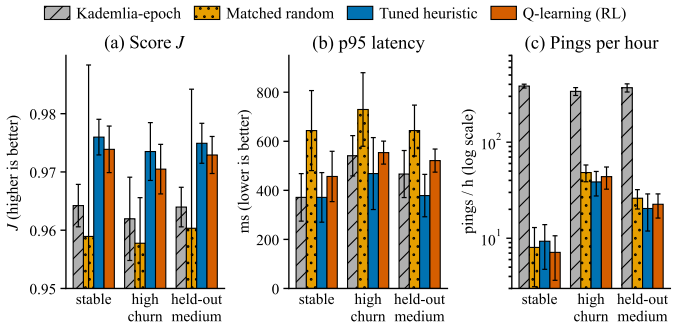


Figure 1. Mean score J, p95 latency and maintenance pings per hour (logarithmic axis) for each scenario; Q-learning is shown in red. Error bars show one standard deviation across the 20 evaluation seeds.

Agent ablations. Permuting rewards across an episode's transitions lowers J by about 0.008–0.012 relative to an unshuffled control, and $\gamma = 0$ performs worse in all three scenarios. The learned behaviour therefore depends on the reward signal and, to a lesser extent, on multi-step credit assignment. None of the 11 variants reached the tuned heuristic.

Latency-weight sweep. Retraining with a larger w_L, scored under the canonical weights, was associated with lower RL p95 latency (e.g., −131 ms in stable at $w_L = 1.0$) but with 74–100 instead of 8–44 pings/h. In high_churn, RL's p95 remained above the tuned heuristic's. One cell exceeded the tuned heuristic on stable J, but it was selected post hoc from 36 combinations and has not been validated. The sweep does not establish that the latency weight explains the gap.

Memoryless churn. With Weibull shape 1.0, the policy ranking was unchanged, and RL's gap to the tuned heuristic did not shift detectably (+0.0007 [−0.0004, +0.0018]).

6 Conclusions and Limitations

Supported by the primary evaluation. In this simulator, a tabular Q-learner over 540 observable states outperforms Kademlia-style epoch maintenance on J in all three scenarios at far lower maintenance-ping rates, by targeting its decisions rather than merely acting less. **Not supported.** RL does not match the tuned 32-configuration heuristic, which achieves higher J and lower p95 latency. We make no claim about real-world deployments.

Limitations. The study uses a single custom simulator with atomic lookups, perfect storage, 128 nodes and one adapting node. Headroom is small: success without any maintenance is already 98.3–99.5%, so most of the signal lies in latency and cost. The state is hand-binned and tabular under partial observability; there are only 20 evaluation seeds and 3 training runs; and the tuning budget favours the heuristic (32 configurations versus 4 RL grid points). Kademlia-epoch quantises Kademlia's rule to 30 s epochs, and bursty churn was not simulated.

Future work (proposed, not performed). Simulate and evaluate correlated churn; confirm the exploratory leads on fresh, pre-registered seeds; re-tune the heuristic jointly with the reward weights and compare latency–ping Pareto fronts; and test larger networks and non-atomic lookups. Details: `FINAL_RESULTS.md`.

References

[1] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Proc. IPTPS*, 2002.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[4] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[5] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.